

AntiBot/WAF GW 1.9.9

Reverse Proxy with Layered Anti-Automation Defenses + Web
Application Firewall

Implementation Report

Author:	Pedro Tarrinho
Date:	2026-06-27
Build:	<code>AntiBotWaf_GW_1.9.9</code> (multi-arch: amd64 · arm64 · arm/v7)
Stack:	Python 3.13 / aiohttp / Podman (Chainguard distroless) / Cloudflare Tunnel front
Datastore:	SQLite (default) · optional PostgreSQL / TimescaleDB (dual-write)
Purpose:	Defensive control evaluation — implementation + bypass-surface review
Document version:	2.0 (supersedes the 1.0 implementation report)

1. Executive Summary

AntiBot/WAF GW is a reverse HTTP/HTTPS proxy that sits in front of a protected upstream application and filters automated and malicious traffic before it reaches the origin. Version 1.9.9 extends the original six-layer anti-automation design into a full layered defense stack combining **bot/agent detection**, a **request-time Web Application Firewall (WAF)**, **IP / reputation intelligence**, and a **risk-scoring engine** with an operator console of eleven dashboards.

Every request passes through an aiohttp middleware chain. Each layer addresses a distinct attack surface used by CLI tools, scrapers, headless browsers, AI agents, and exploitation frameworks. Detections feed a weighted risk score; once a score crosses the (NAT-aware) ban threshold the identity enters a hostile pool and is served a *silent decoy* — the upstream homepage or a 404 — so the client cannot tell it was caught.

Implemented controls (1.9.9)

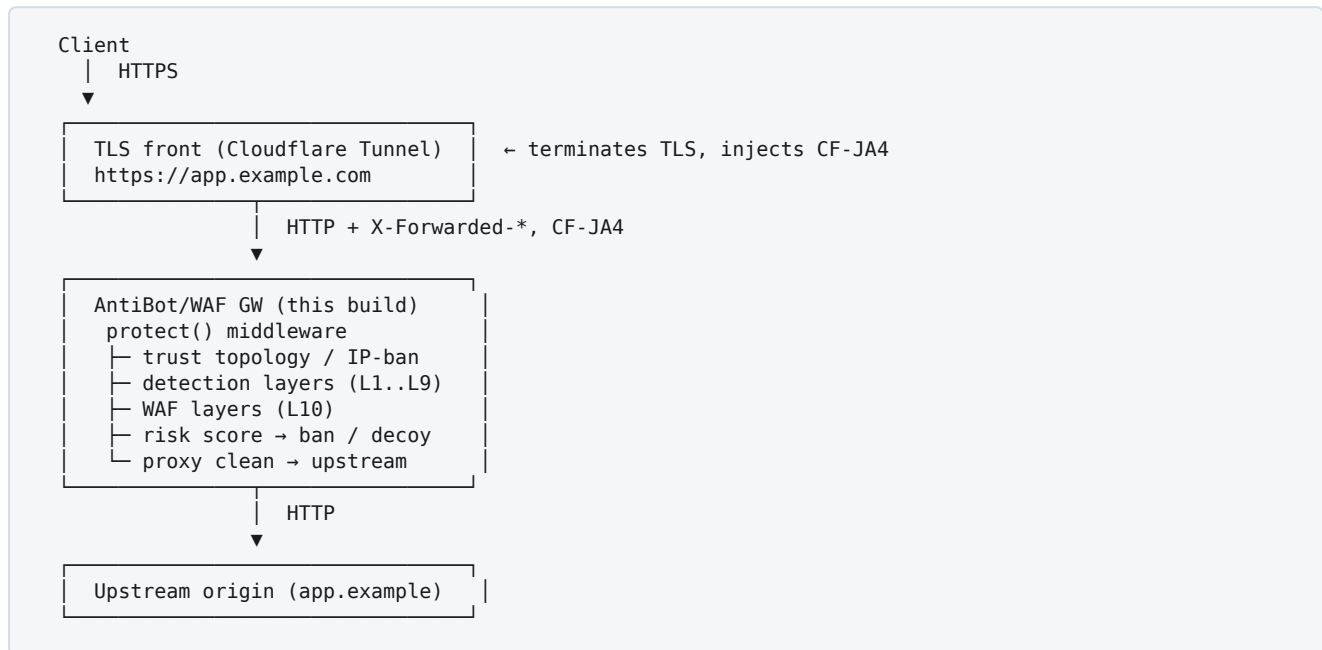
- **36 weighted detection signals** across UA, path, header, behavioural, fingerprint, and reputation families, plus **13 hot-toggable kill-switches** and a decay-aware risk model with NAT-aware thresholds and an optional strict-PoW "Anubis" mode.
- **Request-time WAF**: HTTP request-smuggling, verb-override, header injection (SSTI / host-header), body inspection (XXE, prototype-pollution, critical injection), file-upload, GraphQL abuse, and slow-loris — each behind its own `WAF_*_ENABLED` kill-switch.
- **Reputation & threat intel**: AbuseIPDB, CrowdSec, MaxMind geo/ASN, Tor exit list, and the Feodo / CINS / URLhaus C2 + malware feeds.
- **Challenge stack**: heuristic JS cookie gate, optional Cloudflare Turnstile, proof-of-work, and an AI-labyrinth / tarpit for crawlers that follow hidden links.
- **JA4 TLS-fingerprint binding** of the challenge cookie when a JA4-injecting front (cloudflared) is present.
- **Ed25519 gateway mesh**: peer gateways exchange signed reputation offers; only offers verifying against a registered peer public key are applied.
- **196 hot-reload knobs**, all tunable at runtime via `/secured/config` and the operator dashboards without a restart.
- **Dual datastore** (SQLite default, optional PostgreSQL/TimescaleDB with dual-write) for bans, config, audit, and observational metrics that survive restarts and session-key rotation.

The build ships as a multi-arch container image (amd64 / arm64 / arm/v7) with a clean CVE posture (0 CRITICAL / 0 HIGH on all three arches) and is validated by a 20-stage release pipeline covering unit/functional/integration tests, mutation testing, SAST, dependency/image CVE scans, dynamic security testing, and a written per-release audit record.

2. Architecture

2.1 Deployment topology

The gateway runs as a single container behind a TLS-terminating front (Cloudflare Tunnel in the reference deployment, which also injects the client `CF-JA4` TLS fingerprint). The front forwards to the gateway's listener; the gateway applies all detection + WAF layers and proxies clean traffic to the upstream origin.



2.2 Request lifecycle

Every request flows through the `protect()` middleware. Short-circuit checks (IP-ban, admin-path gating, bypass paths, authorized-bot pass-through) run first; then the detection + WAF layers accumulate risk; finally the risk verdict selects forward / challenge / silent-decoy / ban.

```

Request → protect() middleware
1. Source-IP resolution (XFF trust topology) + persistent IP-ban check
2. Admin-path gating (admin-IP allowlist + session) / authorized-bot rules
3. Detection layers L1..L9 (accumulate weighted risk)
4. WAF layers L10 (smuggling / verb / header / body / upload / graphql / slowloris)
5. Challenge gate (JS cookie / PoW / Turnstile) when required
6. Risk verdict:
   score < SOFT_CHALLENGE_SCORE → forward to upstream
   soft band → monitor / challenge
   score ≥ RISK_BAN_THRESHOLD → hostile pool + silent decoy
7. Rate-limit (per-IP + per-identity token buckets, global RPS cap)
8. Forward clean traffic → upstream; record event → datastore
  
```

2.3 State model

Hot-path state (per-identity risk, token buckets, session/fingerprint maps, recent events, timeline buckets) lives in memory for zero-latency reads. Durable state (bans, `config_kv`, secrets, admin-IP ACL, audit, observational metrics) is persisted to the datastore and rehydrated on boot. Security-critical ban state is

rehydrated *synchronously* before the server accepts traffic; cosmetic dashboard state is rehydrated in a background task so the gateway accepts connections within seconds of a restart. The persistent `ip_bans` table keys on the raw IP and therefore survives session-cookie and fingerprint rotation.

3. Protection Layers

The layers below are grouped by attack surface. Each weighted signal is independently hot-toggleable and contributes to the identity's risk score; WAF layers are terminal (immediate silent-decoy on a confirmed match).

1 User-Agent & client identity

Blocklist of 60+ UA fragments (HTTP libraries, scanners, headless browsers, and AI crawler bots such as `gptbot`, `claudebot`, `anthropic`), plus empty / too-short / non-browser / platform-mismatch signals against the declared `Sec-CH-UA-Platform`.

2 Path & honeypot traps

Suspicious-path regex (VCS metadata, backup files, traversal, SQLi/XSS/LFI markers), advertised honeypot paths with no upstream (`/.git/HEAD`, `/.env`, `/actuator/env`, ...), hidden bot-trap form fields, and an AI-labyrinth that bans clients which follow injected `rel=nofollow` links.

3 Header completeness & consistency

Required-header allowlist, `Accept-*` / `Sec-Fetch-*` presence, Sec-Fetch navigation coherence, Client-Hints vs UA consistency, and an HTTP/2 SETTINGS fingerprint check.

4 Behavioural analysis

Inter-arrival timing regularity, path-enumeration thresholds, HTML-without- assets ratio, and request-cadence autocorrelation — robotic-pattern signatures.

5 Challenge & proof-of-work

Heuristic JS cookie gate (auto-mint on first qualifying HTML GET), optional Cloudflare Turnstile, and a proof-of-work gate (strict "Anubis" mode for configured paths). Unsolved challenges are silently decoyed.

6 Rate limiting

Per-IP and per-identity token buckets plus a global RPS cap; throttled traffic receives HTTP 429 with no risk penalty (pure back-pressure).

7 Canary & referer integrity

Canary tokens planted in responses (echo detection for LLM agents that replay page content) and referer-chain validation (ghost / loop signals).

8 TLS / JA4 fingerprint

JA3/JA4 handshake fingerprints matched against a deny-list; the challenge cookie is bound to the observed JA4 so it cannot be replayed under a different TLS stack.

9 Reputation & threat intelligence

AbuseIPDB confidence tiers, CrowdSec decisions, MaxMind geo/ASN (country allow/deny + hosting-ASN signal), Tor exit detection, and the Feodo / CINS / URLhaus C2 + malware feeds.

10 Web Application Firewall

Request-time WAF inspecting method, headers, and (bounded) body: HTTP request-smuggling, verb-override, header injection (SSTI / host-header), XXE, prototype-pollution, critical command/SQL injection, malicious file-upload, GraphQL introspection/batching abuse, and slow-loris. Each detector is gated by its own `WAF_*_ENABLED` kill-switch (default-on, per-vhost overridable) and yields a silent decoy on a confirmed match.

Cross-cutting: a decay-aware weighted risk model (`SOFT_CHALLENGE_SCORE` / `RISK_BAN_THRESHOLD` / `RISK_BAN_THRESHOLD_NAT` / `RISK_DECAY_HALFLIFE_SECS` / `HOSTILE_BAN_SECS`); silent-decoy responses (`BLOCK_RESPONSE_MODE` = homepage | 404); a persistent raw-IP hostile pool; and an Ed25519-signed gateway mesh for sharing reputation between peers.

4. Configuration

4.1 Key environment variables

Variable	Example	Purpose
<code>UPSTREAM</code>	<code>https://app.example.com</code>	Origin the gateway proxies to (required)
<code>LISTEN_HOST</code> / <code>LISTEN_PORT</code>	<code>0.0.0.0 / 8443</code>	Gateway bind address
<code>TRUST_XFF</code>	<code>last</code>	X-Forwarded-For trust topology (off / last / all / N)
<code>JA4_TRUSTED_PEERS</code>	<code><front IP/CIDR></code>	Source allowed to inject the JA4 header
<code>BLOCK_RESPONSE_MODE</code>	<code>homepage</code>	Decoy served to banned clients (homepage 404)
<code>SOFT_CHALLENGE_SCORE</code> / <code>RISK_BAN_THRESHOLD</code>	<code>4 / configurable</code>	Risk bands (hot-reloadable)
<code>POSTGRES_DSN</code>	(unset = SQLite)	Enables PostgreSQL/TimescaleDB dual-write
<code>WAF_*_ENABLED</code>	<code>1</code>	Per-detector WAF kill-switches (default-on)
<code>VHOST_PORT_AWARE</code>	<code>0</code>	1.9.9 — treat the Host's port as part of the vhost identity, so <code>host:8008</code> and <code>host:8009</code> are distinct vhosts (separate upstream/policy/stats/bans). Default off (host-only). Front must forward the original <code>host:port</code> .

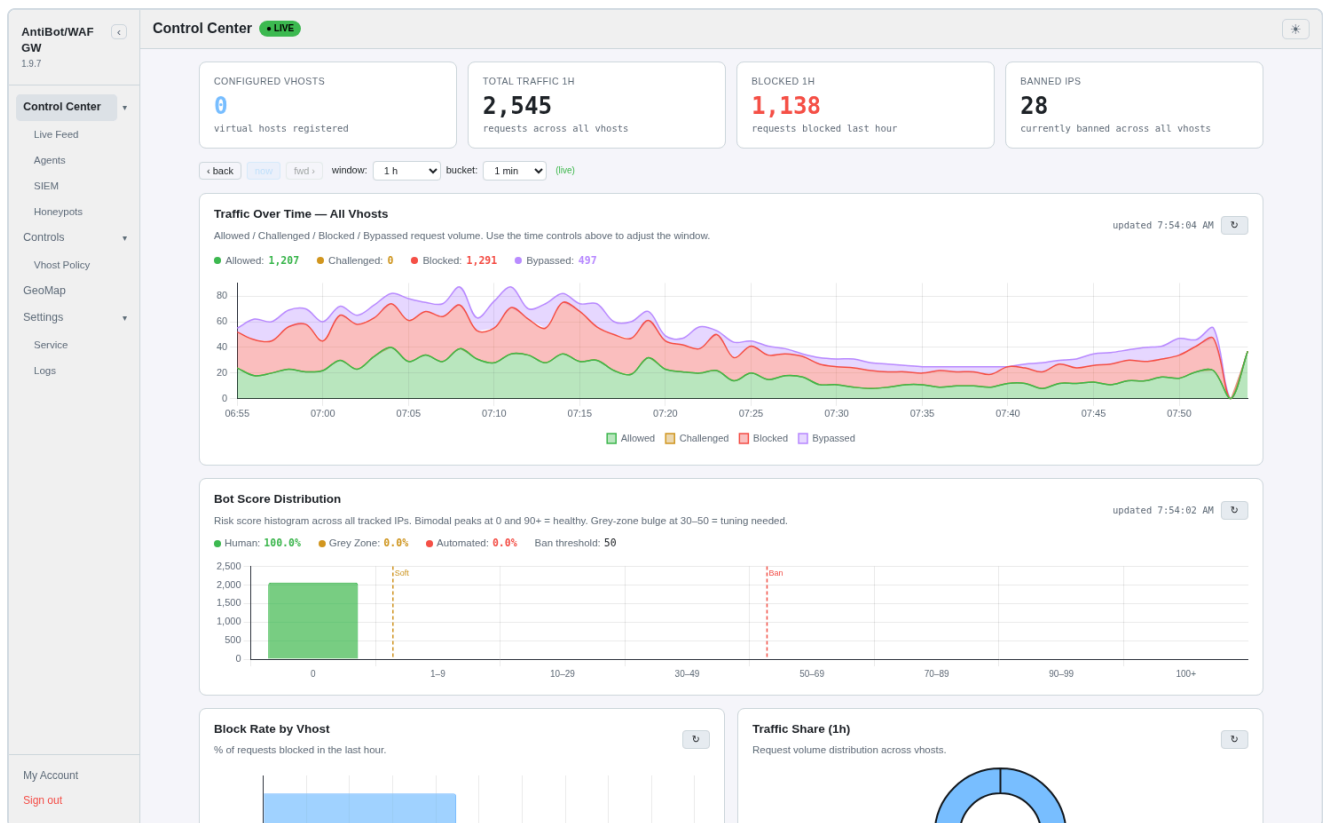
All 196 hot-reload knobs are readable / settable at runtime; trust-topology knobs (`TRUST_XFF` , `TRUSTED_PROXIES`) are env-pinned and cannot be changed from the database.

4.2 Operator & internal endpoints

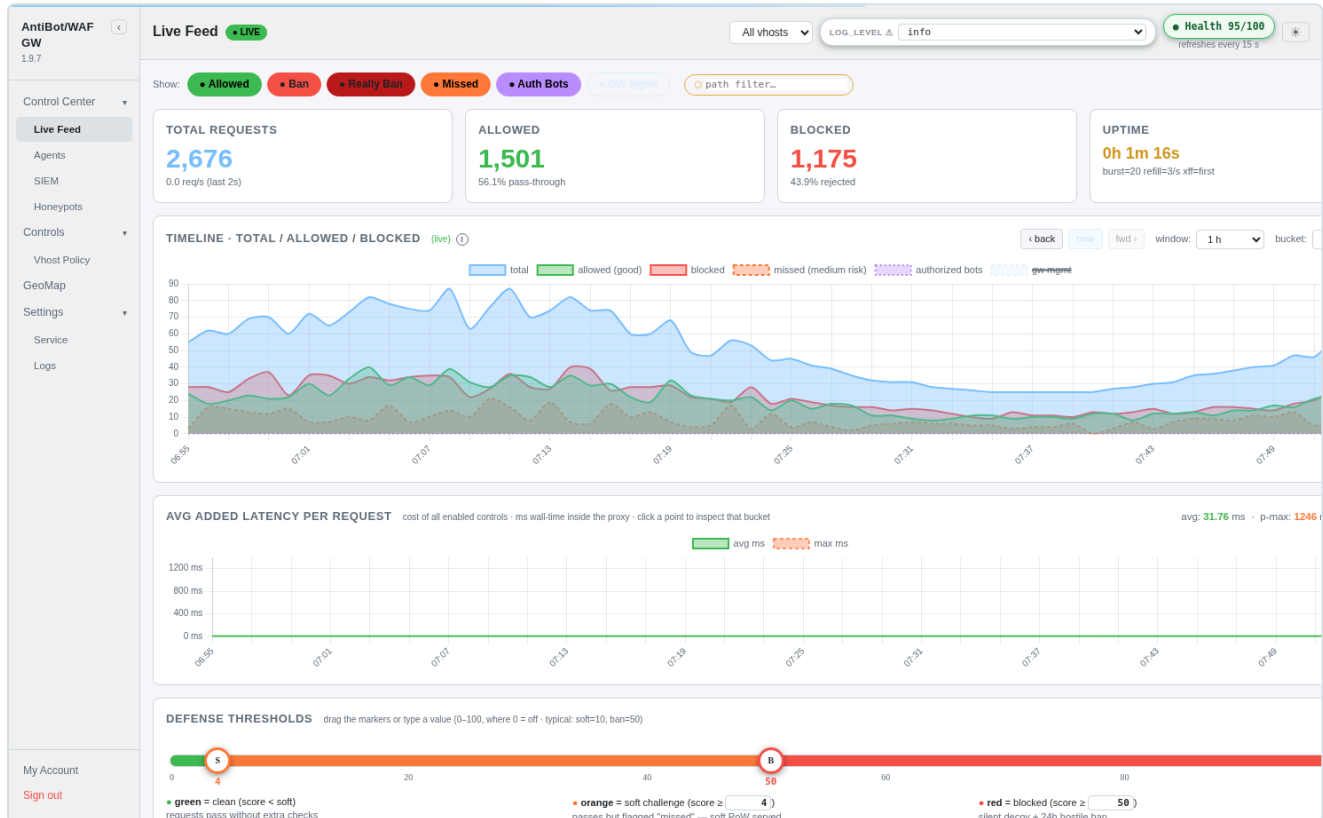
The operator console lives under `/antibot-appsec-gateway/secured/*` (eleven dashboards: live-feed, control-center, agents, controls, settings, honeypots, geo, logs, service, siem, vhost-policy). Admin endpoints are gated by an admin-IP allowlist plus an authenticated session; unauthenticated requests receive an upstream-style 404 (the route's existence is never confirmed). The master light/dark theme is set via `POST /secured/ui-theme` and is honoured by every dashboard.

4.3 Operator console — dashboards

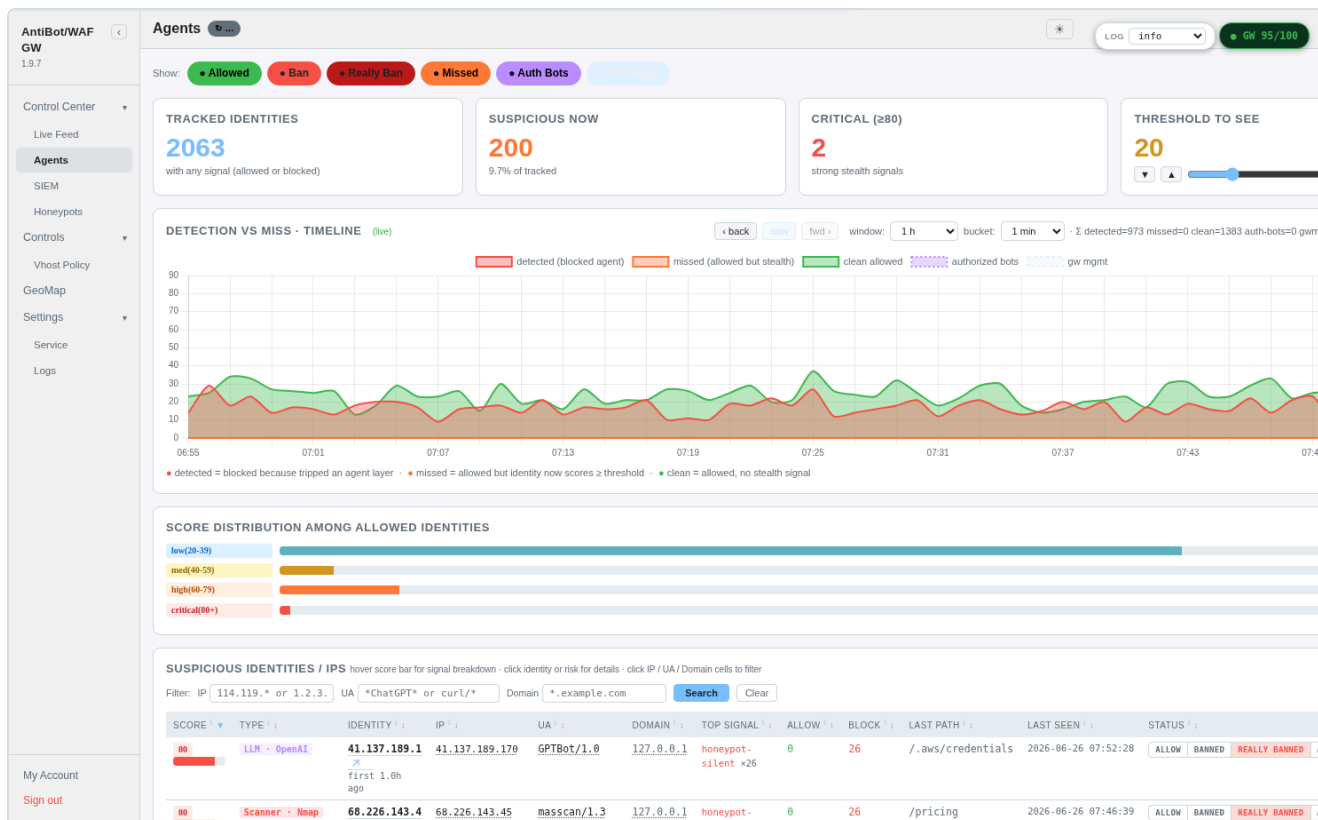
The operator console comprises eleven dashboards under `/antibot-appsec-gateway/secured/*`. The captures below are from a reference instance fronting a generic upstream (`https://example.com`) with synthetic mixed traffic (legitimate browsers, blocked bots, honeypot probes, and banned scanners).



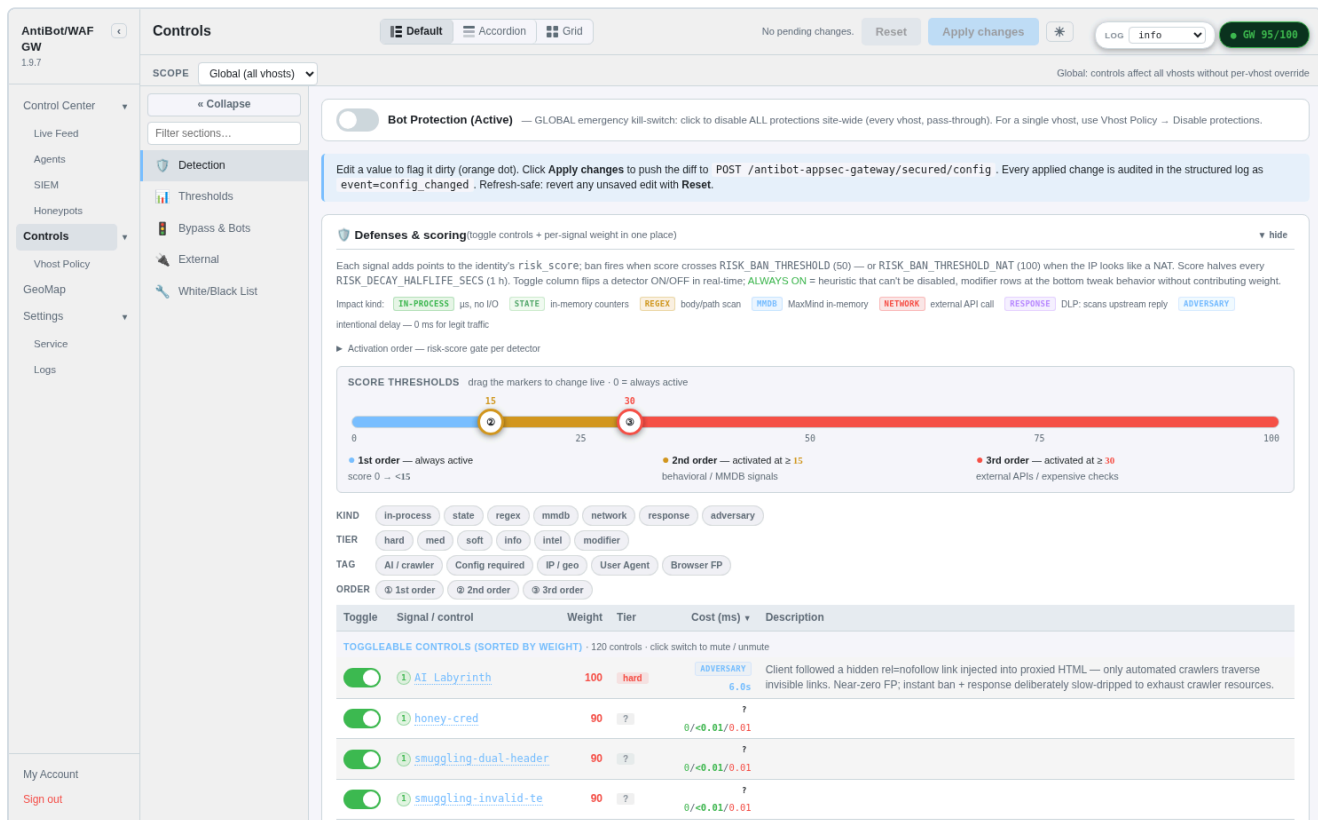
Control Center — post-login landing: vhost traffic summary, banned-IP count, Traffic-Over-Time, and the Bot Score Distribution histogram (Soft / Ban thresholds marked).



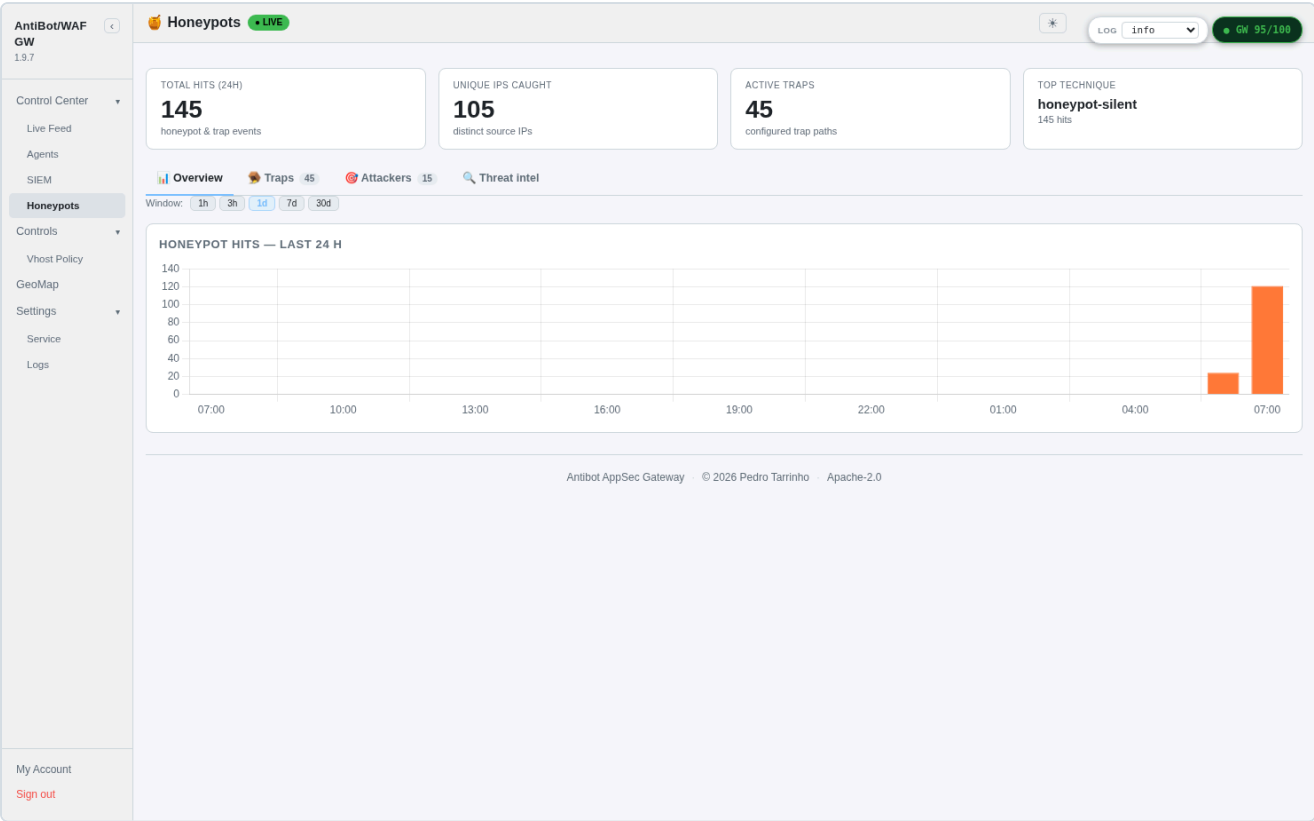
Live Feed — real-time per-request stream (Allowed / Blocked / Missed / Auth-bots / GW-mgmt), with the reason-details drill-down and inline ban / unban actions.



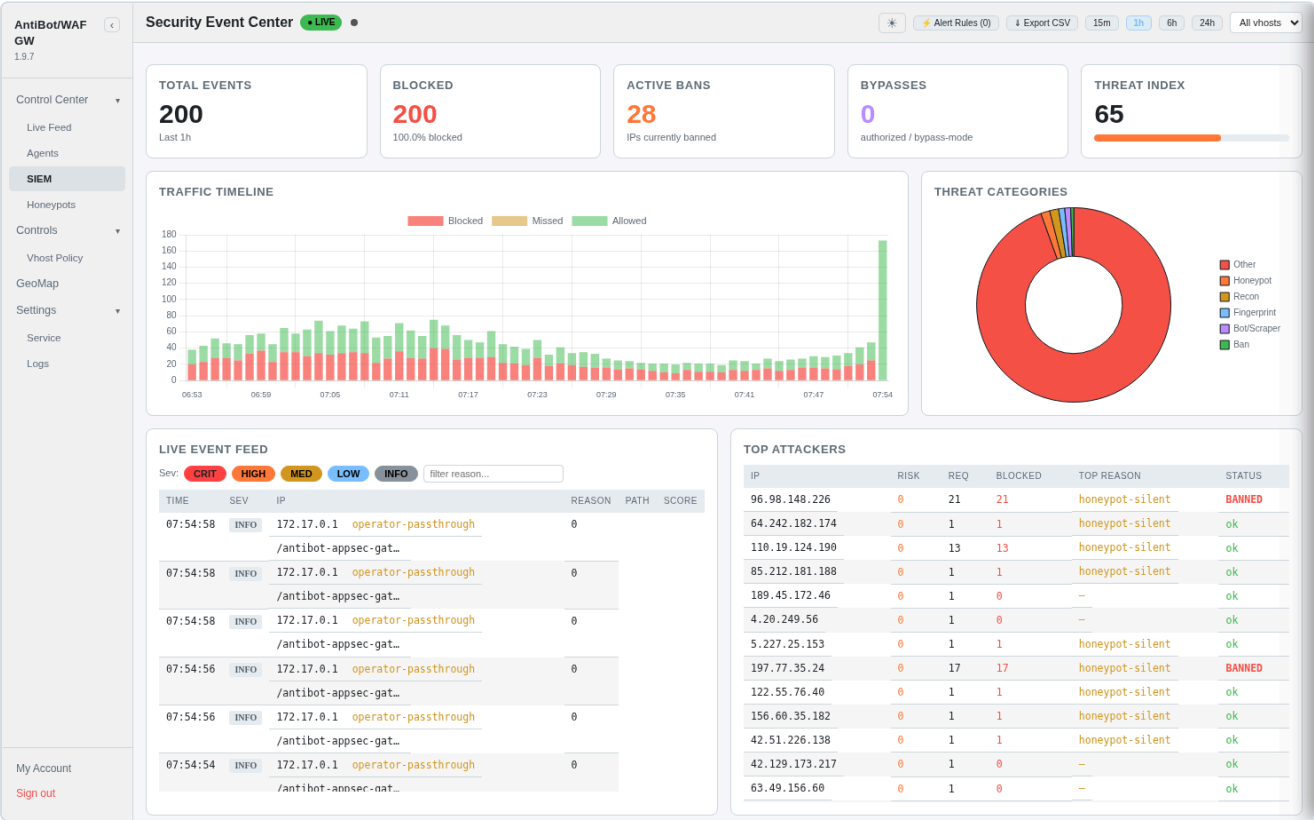
Stealth Agent Hunter — identities that passed every block but show stealth signals; per-identity stealth score with IP / UA / session / JA4 / timing popover and per-signal breakdown.



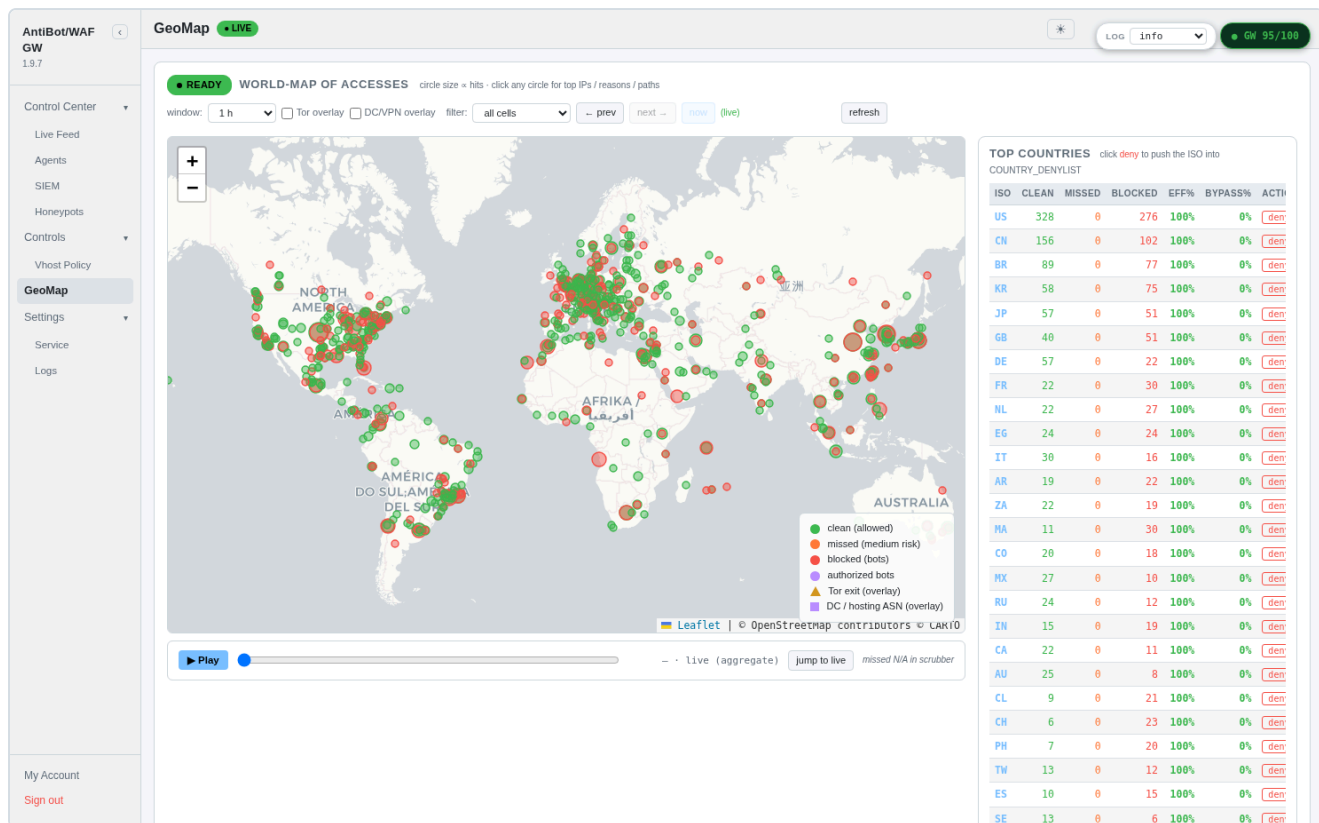
Controls — every hot-reload knob (toggles, thresholds, lists), the merged Defenses & scoring table, strict-PoW (Anubis) toggle, and the admin-IP allowlist.



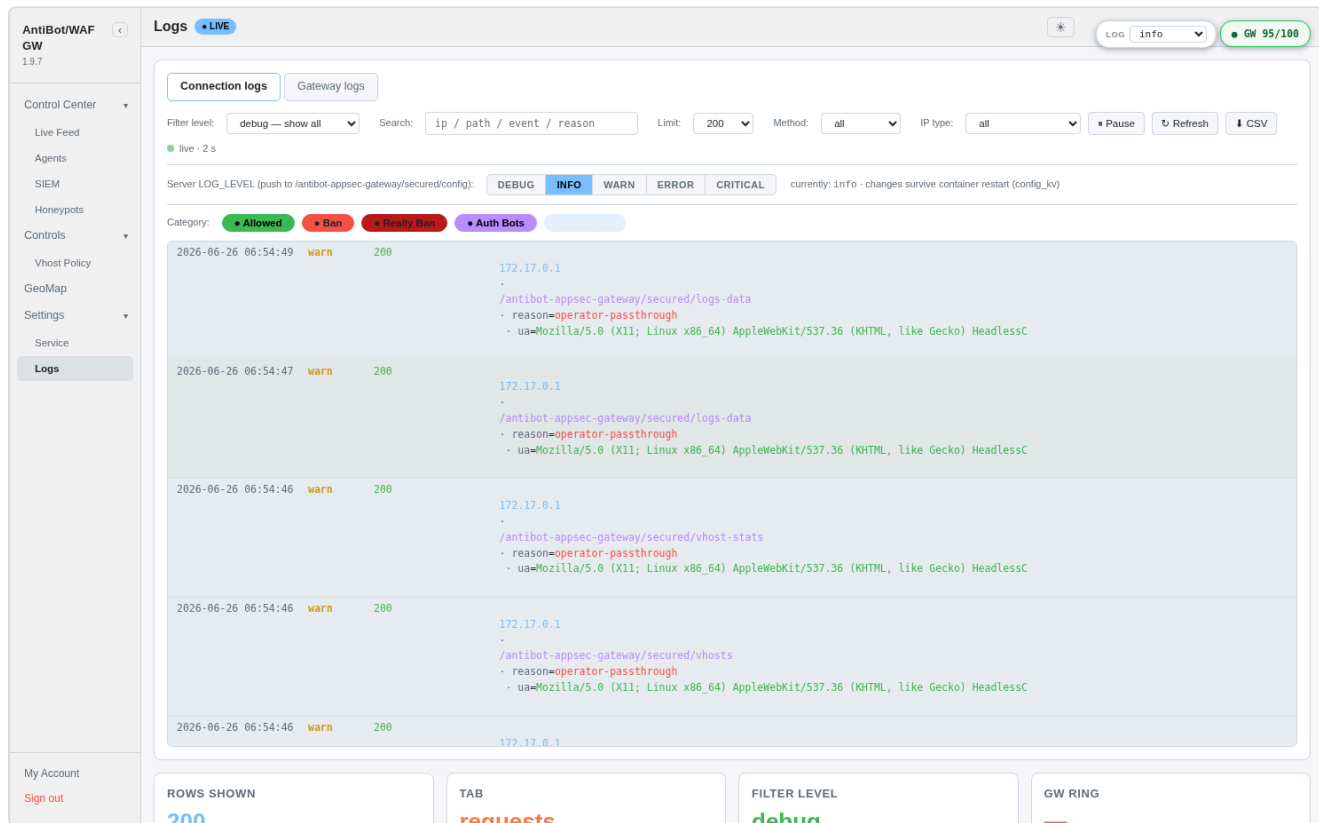
Honeyspots — trap-catch analytics, the Attack Playbook (catches grouped by technique, scanner-tool fingerprinting, predicted next probes), and honeypot-path suggestions.



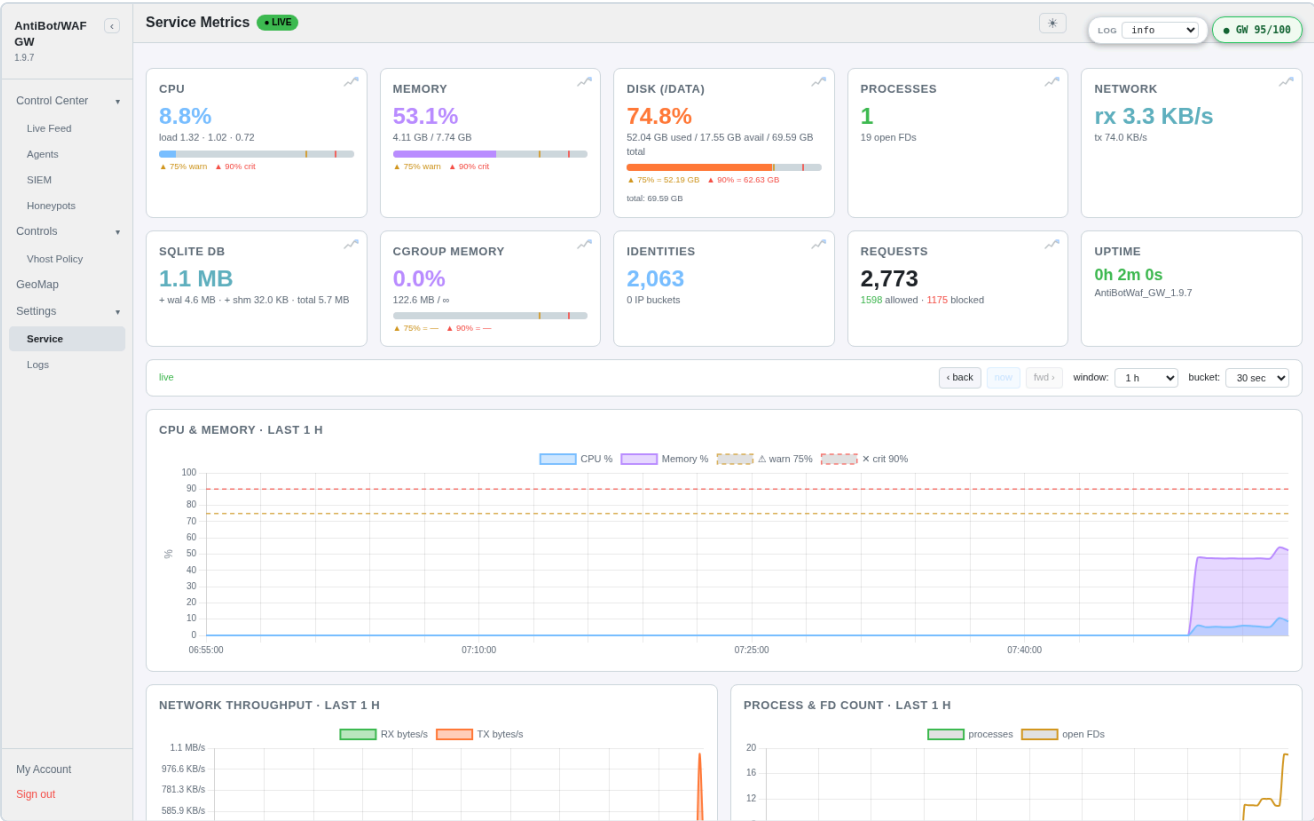
SIEM — alert rules + fired-alert timeline for exporting security events.



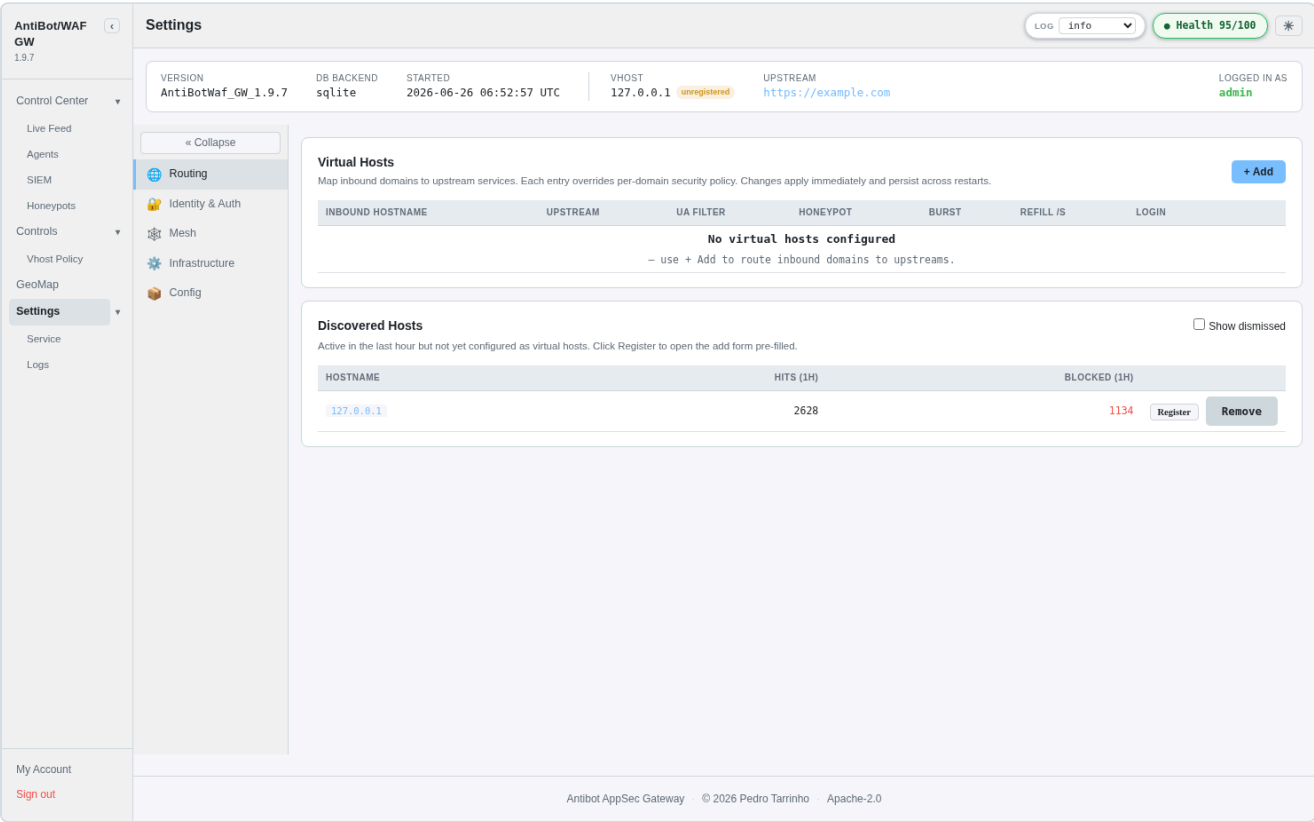
GeoMap — world map of accesses by source country/ASN (MaxMind GeoLite2), with clean / blocked / Tor-exit / hosting overlays and a Top-Countries breakdown; the reference instance shows blocked probes from across the Americas, Europe, Africa, Asia, and Oceania.



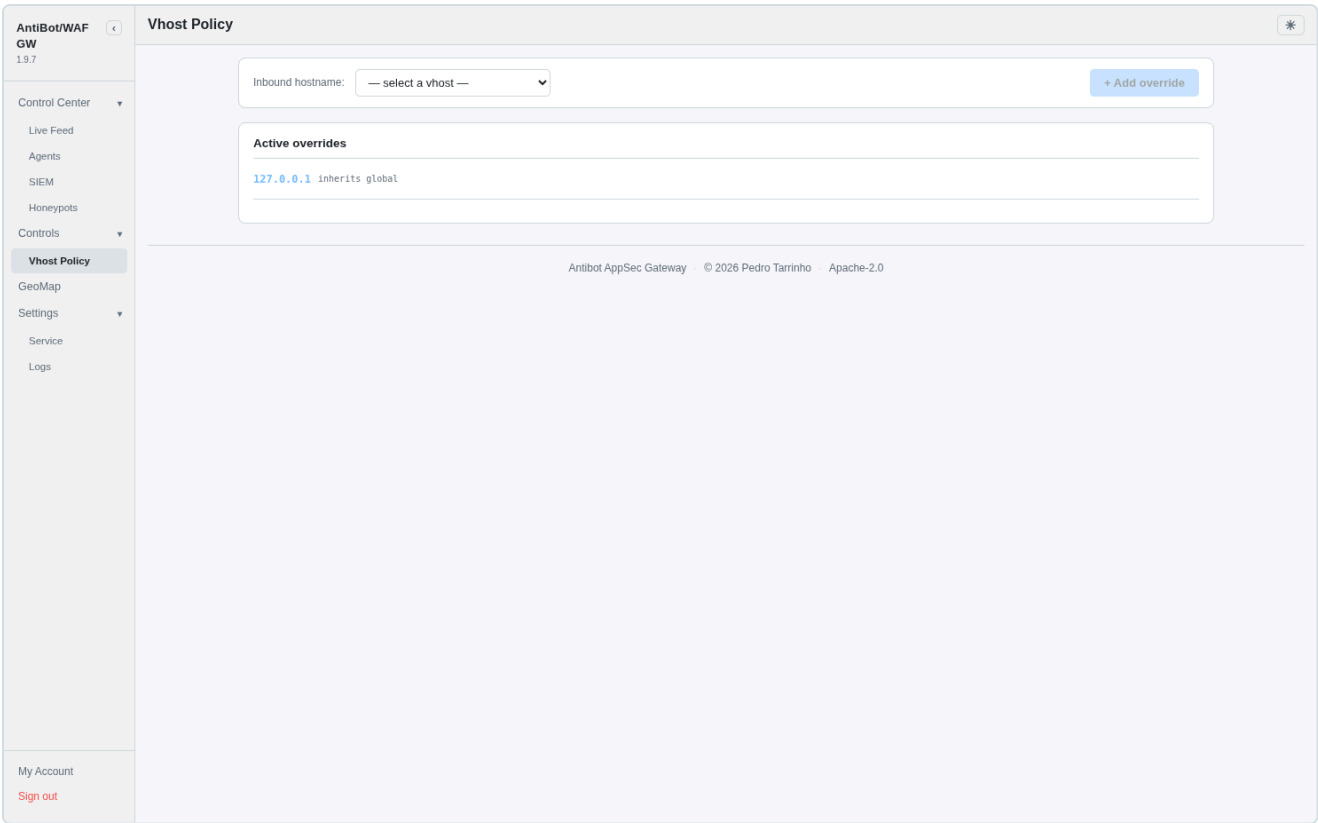
Logs — searchable structured request log with reason / status / vhost filters and export.



Service Metrics — gateway health over time: request throughput, latency, DB size, and resource counters sampled on a schedule.



Settings — routing (virtual hosts + discovered hosts), identity & auth, mesh, infrastructure integrations, and the config / backup tools.



Vhost Policy — per-virtual-host security-policy overrides layered over the global defaults.

5. Test Results

Validation is driven by a 20-stage release pipeline (`rules.md`). The 1.9.9 release record is held in `validation/1.9.8.md` .

Stage	Result
Unit / functional / integration / component / regression	1184 passed (SQLite gate)
Static analysis (Bandit / Semgrep / ruff)	0 High/Critical; lint blocking categories clean
Image CVE scan (Trivy ×3 arches)	0 CRITICAL / 0 HIGH
Dynamic security testing (DAST harness)	open-redirect, SSRF markers, CRLF/header-injection — all defended
Multi-arch build + registry push	amd64 · arm64 · arm/v7 manifest verified

Representative request outcomes observed end-to-end against the live upstream:

Request	Layer	Result
Browser with valid challenge cookie	—	200 forwarded
UA <code>curl/8</code> / <code>python-requests</code>	UA filter	decoy ua-blocked
UA <code>ClaudeBot</code> / <code>GPTBot</code>	UA filter	decoy ua-blocked
Probe <code>/.git/HEAD</code> , <code>/.env</code>	Honeypot	decoy + risk bump
Burst beyond token bucket	Rate limit	429
PoW-required path, no token	Challenge	402 + challenge
XXE / prototype-pollution body	WAF	decoy suspicious-body
Banned raw IP (hostile pool)	IP-ban	decoy ip-ban (every path)

6. Bypass Surface & Residual Risk

The accepted-risk register (re-confirmed each release in `validation/<version>.md` §11b) records design trade-offs that are not defects but carry residual exposure:

ID	Residual risk	Mitigation
R1	<code>agw_csrf</code> cookie is JS-readable (HttpOnly off)	Double-submit token + <code>SameSite=Strict</code>
R2	<code>ALLOW_PRIVATE_UPSTREAM</code> default-allow (SSRF reach)	Operator-scoped; SSRF guard on outbound URLs; env-pinnable
R3	Admin surface hidden behind a decoy-404	Admin-IP allowlist is the real gate (obscurity is a layer, not the control)

Front-of-proxy trust boundary: the gateway's source-IP and JA4 guarantees depend on the front. It must terminate TLS, inject a trustworthy JA4, and overwrite (not append) the client-supplied `X-Forwarded-For`. A front that passes through client-controlled XFF undermines per-IP bans and geo/reputation lookups; pin the trusted front via `TRUST_XFF` + `JA4_TRUSTED_PEERS`.

7. Recommendations

1. Deploy behind a JA4-injecting, XFF-overwriting front (Cloudflare Tunnel or an nginx JA4 build) and pin it with `TRUST_XFF=last` + `JA4_TRUSTED_PEERS`.
2. Keep the WAF kill-switches default-on; disable a layer only per-vhost when a legitimate client trips it, and re-enable after tuning.

3. Enable PostgreSQL/TimescaleDB for multi-replica or long-retention deployments; SQLite is sufficient for single-instance.
4. Configure reputation feeds (AbuseIPDB / CrowdSec) and the C2/malware feeds for environments exposed to broad internet scanning.
5. Set `LOG_RETENTION` / DLP redaction in line with the privacy posture — IP addresses are PII; prune the events store on schedule.
6. For mesh deployments, upgrade all peers together (the Ed25519 model is not compatible with the legacy symmetric keys) and rotate keypairs.

8. Appendix

8.1 Run

```
cd /opt/antibot-gw

# Container (recommended) – multi-arch image from your registry
podman run -d --name antibot-gw \
  -e UPSTREAM=https://app.example.com \
  -e TRUST_XFF=last \
  -p 8443:8443 \
  <your-registry>/antibotappsecgw:1.9.9

# Front the listener with a TLS tunnel that injects JA4 (e.g. cloudflared)
cloudflared tunnel --url http://127.0.0.1:8443
```

8.2 Selected file inventory

Path	Role
<code>proxy.py</code>	Application entrypoint + route table + startup/rehydrate
<code>core/proxy_handler.py</code>	<code>protect()</code> middleware, detection + WAF wiring, silent-decoy, dashboards data
<code>config.py</code>	Environment variables, knob defaults, version string
<code>vhost.py</code>	Per-vhost effective-config accessor (<code>vc()</code>) + coercion
<code>scoring.py</code> / <code>identity.py</code>	Risk model, ban/unban, identity + session fingerprinting
<code>detection/</code> · <code>reputation/</code> · <code>integrations/</code>	Signal families, feeds, JA4/Redis/JWT integrations
<code>db/sqlite.py</code> · <code>db/postgres.py</code>	Dual-backend persistence + dual-write
<code>admin/</code> · <code>dashboards/</code>	Auth, mesh, settings + the eleven operator dashboards

8.3 Verdict

AntiBot/WAF GW 1.9.9 implements a defense-in-depth stack — bot/agent detection, a request-time WAF, reputation intelligence, and a decay-aware risk model — with a clean CVE posture and a complete, version-controlled validation record. The dominant residual risk is the trust boundary at the front of the proxy; the documented deployment guidance closes it.

Author: Pedro Tarrinho · AntiBot/WAF GW 1.9.9 · Document version 2.0 · 2026-06-27. Targets and hostnames in this document are illustrative.